

Chapter 4 - CORE APIs (Questions and Answers)

Pregunta número 1

1. What is output by the following code? (Choose all that apply.)

```
1: public class Fish {  
2:     public static void main(String[] args) {  
3:         int numFish = 4;  
4:         String fishType = "tuna";  
5:         String anotherFish = numFish + 1;  
6:         System.out.println(anotherFish + " " + fishType);  
7:         System.out.println(numFish + " " + 1);  
8:     } }
```

- A. 4 1
- B. 5
- C. 5 tuna
- D. 5tuna
- E. 51tuna
- F. The code does not compile.

Answer 1: Considero que esta pregunta es una trampa clásica en la que, si leemos rápido, nos ponemos directamente a calcular la salida sin revisar si el código compila. Hagamos un recorrido línea por línea. En la línea 3 tenemos `int numFish = 4`, todo perfecto. En la línea 4 tenemos `String fishType = "tuna"`, que también es correcto. El problema aparece en la línea 5: `String anotherFish = numFish + 1`. Aquí debemos recordar que el operador `+` solo se convierte en concatenación cuando al menos uno de los operandos es un `String`. En este caso, tanto `numFish` como el literal `1` son de tipo `int`, por lo que Java realiza una suma aritmética y el resultado es el `int` 5. Después intentamos guardar ese `int` dentro de una variable de tipo `String`, y Java no hace ninguna conversión automática de un primitivo a `String` en una asignación, por lo que el compilador lanza un error de tipos incompatibles (`int cannot be converted to String`). Por lo tanto, la respuesta correcta es el inciso F. Si revisamos los distractores, vemos que están diseñados para que nos enfoquemos en la salida: si el código compilara, la línea 6 imprimiría "5 tuna" (inciso C) y en la línea 7, como la expresión se evalúa de izquierda a derecha, `numFish + " "` se convierte primero en el `String` "4 " y después se le concatena el 1, imprimiendo "4 1" (inciso A). Es decir, los incisos A y C serían la salida si la línea 5 fuera válida, pero como no compila, ninguno se imprime. Los incisos B, D y E son variaciones que juegan con la confusión entre suma y concatenación. Los puntos clave que me gustaría resaltar son: antes de calcular cualquier salida, debemos verificar que cada asignación tenga tipos compatibles; un `int` nunca se puede asignar directamente a

un String (necesitaríamos algo como `String.valueOf(numFish + 1)`). Por otro lado, el operador `+` se evalúa de izquierda a derecha: mientras no aparezca un String, se suma; en cuanto aparece un String, todo lo que sigue se concatena.

Pregunta número 2

2. Which of these array declarations are not legal? (Choose all that apply.)
- A. `int[][] scores = new int[5][];`
 - B. `Object[][][] cubbies = new Object[3][0][5];`
 - C. `String beans[] = new beans[6];`
 - D. `java.util.Date[] dates[] = new java.util.Date[2][];`
 - E. `int[][] types = new int[];`
 - F. `int[][] java = new int[][];`

Answer 2: En esta pregunta nos piden lo contrario a lo que normalmente buscamos: tenemos que identificar las declaraciones que no son legales. La regla de oro aquí es que, cuando creamos un arreglo con `new`, estamos obligados a indicar el tamaño de al menos la primera dimensión (la de más a la izquierda), a menos que usemos una lista de inicialización con llaves `{}`; además, el tipo que va después de `new` debe ser un tipo de dato real. Empezando por el inciso A (`int[][] scores = new int[5][];`), esta declaración es legal, ya que solo indicamos el tamaño de la primera dimensión y dejamos la segunda vacía; esto crea lo que se conoce como un arreglo irregular (jagged array), donde cada fila se puede crear después con el tamaño que queramos. El inciso B (`Object[][][] cubbies = new Object[3][0][5];`) se ve muy extraño, pero también es legal: tenemos tres pares de corchetes en ambos lados y un tamaño de 0 es completamente válido en Java. El primer inciso incorrecto, y por lo tanto parte de la respuesta, es el C (`String beans[] = new beans[6];`). Aquí está la trampa: `beans` es el nombre de la variable, no un tipo de dato, por lo que el compilador no encuentra ningún tipo llamado `beans` y marca un error (cannot find symbol); lo correcto sería `new String[6]`. El inciso D (`java.util.Date[] dates[] = new java.util.Date[2][];`) es legal, ya que Java nos permite repartir los corchetes antes y después del nombre de la variable, y en total suman dos dimensiones; el nombre totalmente calificado `java.util.Date` también es válido. El inciso E (`int[][] types = new int[];`) no compila porque no indicamos ningún tamaño, y además estamos intentando asignar un arreglo de una dimensión a una variable de dos dimensiones. Finalmente, el inciso F (`int[][] java = new int[][];`) tampoco compila, ya que, aunque las dimensiones coinciden, no indicamos el tamaño de la primera dimensión. Con lo anterior dicho, las respuestas correctas son los incisos C, E y F. Los puntos clave son: al usar `new` con un arreglo, siempre debemos indicar el tamaño de la primera dimensión o dar una lista de inicialización, pero nunca ambas cosas a la vez; los corchetes pueden ir antes o después del nombre de la variable y se suman; y el tipo que va después de `new` tiene que ser un tipo real, no el nombre de una variable.

Pregunta número 3

3. Note that March 13, 2022 is the weekend when we spring forward, and November 6, 2022 is when we fall back for daylight saving time. Which of the following can fill in the blank without the code throwing an exception? (Choose all that apply.)

```
var zone = ZoneId.of("US/Eastern");  
var date = _____;  
var time = LocalTime.of(2, 15);  
var z = ZonedDateTime.of(date, time, zone);
```

- A. `LocalDate.of(2022, 3, 13)`
- B. `LocalDate.of(2022, 3, 40)`
- C. `LocalDate.of(2022, 11, 6)`

- D. `LocalDate.of(2022, 11, 7)`
- E. `LocalDate.of(2023, 2, 29)`
- F. `LocalDate.of(2022, MonthEnum.MARCH, 13);`

Answer 3: Aquí tenemos el tema del horario de verano (daylight saving time) combinado con la validación de fechas. Lo primero que debemos tener claro es de dónde pueden venir las excepciones en este código: la clase `LocalDate` valida estrictamente que la fecha exista en el calendario, mientras que `ZonedDateTime.of()` es muy permisivo con las horas que no existen o que se repiten por el cambio de horario. Empezando por el inciso A (`LocalDate.of(2022, 3, 13)`), la fecha es válida. Sabemos que ese día los relojes saltan de las 2:00 a las 3:00, por lo que las 2:15 en realidad no existen. Sin embargo, `ZonedDateTime.of()` no lanza ninguna excepción; simplemente ajusta la hora hacia adelante la duración del hueco (gap), dejando la hora en 3:15 con offset -04:00. Por lo tanto, el inciso A es correcto. El inciso B (`LocalDate.of(2022, 3, 40)`) es incorrecto, ya que ningún mes tiene 40 días y `LocalDate.of()` lanza una `DateTimeException` inmediatamente. El inciso C (`LocalDate.of(2022, 11, 6)`) es correcto: ese día los relojes regresan de las 2:00 a la 1:00, por lo que la hora que se repite es de la 1:00 a la 1:59; las 2:15 ocurren una sola vez y no hay ningún problema. Incluso si la hora fuera ambigua, Java tomaría el offset anterior sin lanzar excepción. El inciso D (`LocalDate.of(2022, 11, 7)`) también

es correcto, ya que es un día completamente normal. El inciso E (`LocalDate.of(2023, 2, 29)`) es incorrecto porque 2023 no es año bisiesto, así que el 29 de febrero no existe y se lanza una `DateTimeException`. Finalmente, el inciso F (`LocalDate.of(2022, MonthEnum.MARCH, 13)`) es una trampa de sintaxis: el enum de los meses en Java se llama `Month`, no `MonthEnum`, por lo que el código ni siquiera compila. Con todo esto presente, las respuestas correctas son los incisos A, C y D. Los puntos clave que podemos rescatar son: `ZonedDateTime` nunca lanza excepción por una hora que no existe o se repite por el horario de verano, sino que la ajusta; las excepciones vienen de valores inválidos en `LocalDate.of()`, como un día 40, un mes 13 o un 29 de febrero en un año no bisiesto. Además, debemos recordar que el enum correcto es `Month` (por ejemplo, `Month.MARCH`).

Pregunta número 4

4. Which of the following are output by this code? (Choose all that apply.)

```
3: var s = "Hello";
4: var t = new String(s);
5: if ("Hello".equals(s)) System.out.println("one");
6: if (t == s) System.out.println("two");
7: if (t.intern() == s) System.out.println("three");
8: if ("Hello" == s) System.out.println("four");
9: if ("Hello".intern() == t) System.out.println("five");
```

- A. one
- B. two
- C. three
- D. four
- E. five
- F. The code does not compile.
- G. None of the above

Answer 4: En esta pregunta se está evaluando el String pool y la diferencia entre comparar referencias (`==`) y comparar contenido con (`equals()`). Vayamos línea por línea. En la línea 3, `s = "Hello"` es un literal, por lo que Java lo guarda en el String pool. En la línea 4, `new String(s)` crea obligatoriamente un objeto nuevo en el heap, con el mismo contenido pero con una referencia distinta. En la línea 5, `"Hello".equals(s)` compara el contenido, que es idéntico, así que es `true` y se imprime "one". En la línea 6, `t == s` compara referencias; `t` es el objeto nuevo y `s` es el del pool, por lo que es `false` y no se imprime "two". En la línea 7, `t.intern()` busca en el pool un String con el mismo contenido y regresa esa referencia, que es exactamente la misma que tiene `s`; por lo tanto, es `true` y se imprime "three". En la línea 8, `"Hello" == s` compara dos literales iguales, y como los literales se reutilizan

desde el pool, ambos apuntan al mismo objeto; es true y se imprime "four". En la línea 9, "Hello".intern() regresa la referencia del pool, pero t sigue siendo el objeto creado con new, así que es false y no se imprime "five". Con lo anterior dicho, las respuestas correctas son los incisos A, C y D. Los incisos B y E son las trampas, porque ambas comparan con t usando ==, y t jamás va a ser la misma referencia que un String del pool. El inciso F es incorrecto, ya que todo el código compila sin problemas (var infiere String en ambos casos), y el G queda descartado. Los puntos clave son: `==` compara referencias y `equals()` compara contenido; los literales se guardan y reutilizan en el pool; `new String()` siempre crea un objeto nuevo fuera del pool; e `intern()` siempre regresa la referencia que vive en el pool.

Pregunta número 5

5. What is the result of the following code?

```
7: var sb = new StringBuilder();  
8: sb.append("aaa").insert(1, "bb").insert(4, "ccc");  
9: System.out.println(sb);
```

- A. abbaaccc
- B. abbaccca
- C. bbaaaccc
- D. bbaaccca
- E. An empty line
- F. The code does not compile.

Answer 5: En este problema, el tema principal es StringBuilder y el encadenamiento de métodos (method chaining). Lo más importante que debemos recordar es que StringBuilder es mutable, por lo que cada método de la cadena modifica el mismo objeto y regresa una referencia a él; así que cada paso trabaja con el estado actualizado del paso anterior. Comencemos en la línea 7 creamos un StringBuilder vacío. En la línea 8, primero append("aaa") deja el contenido como "aaa". Después, `insert(1, "bb")` inserta el texto antes del carácter que está en el índice 1; tenemos "a" + "bb" + "aa", lo que nos da "abbaa". Luego, insert(4, "ccc") trabaja sobre "abbaa", cuyos índices son a(0), b(1), b(2), a(3), a(4); al insertar antes del índice 4 tenemos "abba" + "ccc" + "a", lo que nos da "abbaccca". En la línea 9 se imprime "abbaccca". Por lo tanto, la respuesta correcta es el inciso B. Los distractores están diseñados para atrapar errores de conteo: el inciso A ("abbaaccc") sale de pensar que el segundo insert pone el texto al final o de calcular el índice sobre el texto original; los incisos C y D salen de insertar "bb" en el índice 0. El inciso E sería la trampa para quien piense que los métodos regresan copias y el objeto original se queda vacío, y el inciso F es incorrecto porque el código compila perfectamente. Los puntos clave son: StringBuilder es mutable y en una cadena de métodos todos

operan sobre el mismo objeto; e insert(offset, texto) coloca el texto antes del carácter que está en esa posición, recalculando los índices con el estado actual, no con el original.

Pregunta número 6

6. How many of these lines contain a compiler error? (Choose all that apply.)

```
23: double one = Math.pow(1, 2);  
24: int two = Math.round(1.0);  
25: float three = Math.random();  
26: var doubles = new double[] {one, two, three};
```

- A. 0
- B. 1

- C. 2
- D. 3
- E. 4

Answer 6: En este ejercicio tenemos que conocer de memoria los tipos de retorno de los métodos de la clase Math, porque de eso depende si cada asignación compila o no. Repasemos línea por línea para entender qué está sucediendo. En la línea 23, Math.pow(1, 2) siempre regresa un double, y lo guardamos en un double, así que todo perfecto. En la línea 24 tenemos la primera trampa: Math.round() tiene dos versiones; si recibe un float, regresa un int, pero si recibe un double, regresa un long. El literal 1.0 es un double, por lo que el método regresa un long, y no podemos guardar un long en un int sin un cast explícito; el compilador marca un error por posible pérdida de precisión. En la línea 25, Math.random() regresa un double, y tampoco podemos guardar un double en un float sin un cast, así que esta línea tampoco compila. Finalmente, en la línea 26 tenemos un arreglo de tipo double inicializado con one (double), two (int) y three (float); como int y float caben perfectamente en un double, el compilador hace la promoción implícita (widening)

sin problema, por lo que esta línea, por sí misma, es válida. Con todo esto visto, tenemos dos líneas con error de compilación (24 y 25), así que la respuesta correcta es el inciso C. Los incisos A y B son la trampa para quien piense que `Math.round(1.0)` regresa un `int` o que `Math.random()` se puede guardar en un `float`, y los incisos D y E salen de pensar que la línea 26 o la 23 también fallan. Los puntos clave son: `Math.round(double)` regresa `long` y `Math.round(float)` regresa `int`; `Math.random()`, `Math.pow()`, `Math.floor()` y `Math.ceil()` siempre regresan `double`; y en un inicializador de arreglo se aplican las mismas reglas de promoción que en una asignación normal.

Pregunta número 7

7. Which of these statements is true of the two values? (Choose all that apply.)

2022-08-28T05:00 GMT-04:00

2022-08-28T09:00 GMT-06:00

- A. The first date/time is earlier.
- B. The second date/time is earlier.
- C. Both date/times are the same.
- D. The date/times are two hours apart.
- E. The date/times are six hours apart.
- F. The date/times are 10 hours apart.

Answer 7: Aquí tenemos el tema de las zonas horarias y cómo comparar dos fechas con distinto offset. Considero que la clave para no equivocarnos es no comparar las horas locales a simple vista, sino convertir ambas a la hora universal (GMT/UTC) y después compararlas. Un offset de GMT-04:00 significa que esa zona va 4 horas detrás de GMT, por lo que, para obtener la hora en GMT, le sumamos 4 horas a la hora local. Empezando con la primera fecha, tenemos 05:00 con GMT-04:00, por lo que en GMT serían las $05:00 + 4 = 09:00$. En la segunda fecha tenemos 09:00 con GMT-06:00, por lo que en GMT serían las $09:00 + 6 = 15:00$. Ahora que ambas están en la misma referencia, podemos ver que la primera ocurre a las 09:00 GMT y la segunda a las 15:00 GMT; la primera es la más temprana y la diferencia entre ambas es de 6 horas. Con lo anterior dicho, las respuestas correctas son los incisos A y E. Los distractores están pensados para que caigamos en errores con los signos: el inciso C es la trampa para quien convierte solo la primera fecha a GMT (09:00) y la compara contra la hora local de la segunda, que también dice 09:00; el inciso D sale de aplicar los offsets con el signo equivocado ($05:00 - 4 = 01:00$ y $09:00 - 6 = 03:00$), o de restar únicamente los offsets entre sí ($6 - 4 = 2$); el inciso F sale de sumar los dos offsets ($4 + 6 = 10$), y el inciso B simplemente invierte el orden real de las fechas. Los puntos clave son: para pasar una hora a GMT debemos restar el offset, y como restar un número negativo es

sumar, a una zona GMT-X le sumamos X horas; y nunca debemos comparar fechas de distintas zonas horarias mirando solo la hora local.

Pregunta número 8

8. Which of the following return 5 when run independently? (Choose all that apply.)

```
var string = "12345";  
var builder = new StringBuilder("12345");
```

- A. `builder.charAt(4)`
- B. `builder.replace(2, 4, "6").charAt(3)`
- C. `builder.replace(2, 5, "6").charAt(2)`
- D. `string.charAt(5)`
- E. `string.length`
- F. `string.replace("123", "1").charAt(2)`
- G. None of the above

Answer 8: En este ejercicio estamos evaluando los métodos `charAt()` y `replace()` de `String` y `StringBuilder`, y la regla del índice final exclusivo. Lo primero que debemos notar es que la pregunta dice "when run independently", es decir, cada inciso se evalúa partiendo de los valores originales "12345". Recordemos que los índices van del 0 al 4: '1'(0), '2'(1), '3'(2), '4'(3), '5'(4). El primer inciso correcto es el A, ya que `builder.charAt(4)` regresa el carácter del último índice, que es '5'. El siguiente inciso correcto es el B: en `StringBuilder`, `replace(2, 4, "6")` reemplaza desde el índice 2 hasta el 4 sin incluirlo, es decir, los caracteres "34", por lo que el contenido queda como "1265"; después `charAt(3)` regresa '5'. El inciso C es incorrecto: `replace(2, 5, "6")` reemplaza los índices 2, 3 y 4 ("345"), dejando "126", y `charAt(2)` regresa '6', no 5. El inciso D es incorrecto, ya que `string.charAt(5)` intenta acceder a un índice que no existe (el último es el 4) y lanza una `StringIndexOutOfBoundsException` en tiempo de ejecución. El inciso E es una trampa de sintaxis: en un `String`, `length()` es un método y lleva paréntesis; `length` sin paréntesis es un atributo exclusivo de los arreglos, así que este código no compila. El último inciso correcto es el F: `string.replace("123", "1")` regresa un nuevo `String` "145" y su `charAt(2)` es '5'. Con lo anterior dicho, las respuestas correctas son los incisos A, B y F. Los puntos clave son: en `replace()` de `StringBuilder` y en `substring()`, el índice final es exclusivo; `charAt()` con un índice igual a la longitud lanza excepción; y debemos recordar la diferencia entre `arreglo.length` (atributo) y `texto.length()` (método).

Pregunta número 9

9. Which of the following are true about arrays? (Choose all that apply.)
- A. The first element is index 0.
 - B. The first element is index 1.
 - C. Arrays are fixed size.
 - D. Arrays are immutable.
 - E. Calling `equals()` on two different arrays containing the same primitive values always returns `true`.
 - F. Calling `equals()` on two different arrays containing the same primitive values always returns `false`.
 - G. Calling `equals()` on two different arrays containing the same primitive values can return `true` or `false`.

Answer 9: Aquí tenemos una pregunta de teoría sobre los arreglos en Java, así que hagamos un análisis inciso por inciso para entender mejor las posibles respuestas. El primer inciso correcto es el A, ya que en Java los arreglos siempre empiezan en el índice 0; por eso, el inciso B es incorrecto. El siguiente inciso correcto es el C: una vez que creamos un arreglo, su tamaño queda fijo y no puede crecer ni reducirse; si necesitamos un tamaño dinámico, tenemos que usar algo como un `ArrayList`. El inciso D es incorrecto, ya que los arreglos son mutables: podemos cambiar el valor de cualquiera de sus elementos en cualquier momento (por ejemplo, `numeros[0] = 10`). Debemos tener cuidado de no confundir tamaño fijo con inmutable. De ahí pasamos a la parte más tramposa, que son los incisos E, F y G. Los arreglos no sobrescriben el método `equals()`, por lo que usan la versión heredada de `Object`, que compara referencias, no contenido. Si tenemos dos arreglos distintos (dos objetos diferentes en memoria), `equals()` siempre va a regresar `false`, sin importar que contengan exactamente los mismos valores. Por lo tanto, el inciso F es correcto, y los incisos E y G son incorrectos; el inciso G es especialmente engañoso porque suena a una respuesta "segura", pero el resultado no depende del contenido. Con todo esto visto, las respuestas correctas son los incisos A, C y F. Los puntos clave son: los arreglos empiezan en 0, tienen tamaño fijo pero son mutables; y `arreglo1.equals(arreglo2)` solo compara referencias, por lo que, si queremos comparar el contenido, debemos usar `Arrays.equals()`.

Pregunta número 10

10. How many of these lines contain a compiler error? (Choose all that apply.)

```
23: int one = Math.min(5, 3);  
24: long two = Math.round(5.5);  
25: double three = Math.floor(6.6);  
26: var doubles = new double[] {one, two, three};
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Answer 10: Considero que esta pregunta es la gemela de la pregunta 6, ya que de nuevo tenemos que revisar los tipos de retorno de los métodos de Math, pero esta vez los creadores del examen nos ponen a prueba para ver si buscamos errores donde no los hay. Necesitamos ir línea por línea. En la línea 23, Math.min(5, 3) recibe dos int y regresa un int, que guardamos en un int; todo perfecto. En la línea 24, Math.round(5.5) recibe un double y, como vimos antes, regresa un long, que esta vez sí guardamos en una variable long; compila sin problema. En la línea 25, Math.floor(6.6) regresa un double, y lo guardamos en un double; también es correcto. Finalmente, en la línea 26 creamos un arreglo de tipo double con one (int), two (long) y three (double); los tipos int y long se promueven de forma implícita a double, por lo que el inicializador es válido. Con lo anterior dicho, ninguna línea tiene error de compilación, así que la respuesta correcta es el inciso A. Los demás incisos son trampas para quien piense que Math.round() siempre regresa int o que un long no puede ir dentro de un arreglo double. Los puntos clave son: Math.min() y Math.max() regresan el mismo tipo de sus argumentos; Math.round(double) regresa long; y en un arreglo de tipo double podemos meter cualquier tipo numérico primitivo, ya que todos se promueven a double.

Pregunta número 11

11. What is the output of the following code?

```
var date = LocalDate.of(2022, 4, 3);  
date.plusDays(2);  
date.plusHours(3);  
System.out.println(date.getYear() + " " + date.getMonth()  
    + " " + date.getDayOfMonth());
```

- A. 2022 MARCH 4
- B. 2022 MARCH 6
- C. 2022 APRIL 3
- D. 2022 APRIL 5
- E. The code does not compile.
- F. A runtime exception is thrown.

Answer 11: En este ejercicio, el tema principal es la API de fechas y, sobre todo, qué métodos tiene cada clase. Hagamos un análisis línea por línea. En la primera línea creamos un `LocalDate` con la fecha 3 de abril de 2022; todo perfecto. En la segunda línea, `date.plusDays(2)` es un método válido, aunque su resultado se descarta (lo veremos en un momento). El problema aparece en la tercera línea: ``date.plusHours(3)``. Debemos recordar que un `LocalDate` solo almacena año, mes y día; no tiene ninguna información de la hora, por lo que tampoco tiene métodos para sumar horas, minutos o segundos. El compilador no encuentra el método `plusHours()` en la clase `LocalDate` y marca un error (cannot find symbol). Por lo tanto, la respuesta correcta es el inciso E. Vale la pena revisar los distractores: si quitáramos la línea de `plusHours()`, el código imprimiría 2022 APRIL 3 (inciso C), porque `LocalDate` es inmutable y el resultado de `plusDays(2)` nunca se reasignó a la variable `date`. El inciso D (2022 APRIL 5) es la trampa para quien olvide la inmutabilidad, y los incisos A y B (MARCH) son la trampa para quien piense que los meses empiezan en 0, como en la antigua clase `Calendar`; en la API moderna, el mes 4 es abril. El inciso F es incorrecto, porque el código ni siquiera llega a ejecutarse. Los puntos clave son: `LocalDate` no tiene métodos de horas (`plusHours`, `plusMinutes`, etc.), `LocalTime` no tiene métodos de días, y solo `LocalDateTime` y `ZonedDateTime` tienen ambos; además, todas las clases de `java.time` son inmutables, así que si no reasignamos el resultado, la variable no cambia.

Pregunta número 12

12. What is output by the following code? (Choose all that apply.)

```
var numbers = "012345678".indent(1);  
numbers = numbers.stripLeading();  
System.out.println(numbers.substring(1, 3));  
System.out.println(numbers.substring(7, 7));  
System.out.print(numbers.substring(7));
```

- A. 12
- B. 123
- C. 7
- D. 78
- E. A blank line
- F. The code does not compile.
- G. An exception is thrown.

Answer 12: En este problema tenemos los métodos `indent()`, `stripLeading()` y `substring()` de `String`. Hay que ir línea por línea para entender exactamente qué contiene la variable en cada momento. Empezamos con `"012345678".indent(1)`: El método `indent()` agrega la cantidad de espacios indicada al inicio de cada línea y, algo muy importante, normaliza los saltos de línea y agrega un `\n` al final si no lo tiene. Por lo tanto, `numbers` vale `"012345678\n"`. De ahí nos dirigimos a `stripLeading()`, que elimina únicamente los espacios en blanco del inicio; el salto de línea del final se queda. Ahora `numbers` vale `"012345678\n"`. En la primera impresión, `substring(1, 3)` toma desde el índice 1 hasta el 3 sin incluirlo, es decir, los caracteres '1' y '2', y se imprime 12 (inciso A). En la segunda impresión, `substring(7, 7)` tiene el índice de inicio igual al índice final; esto es completamente válido y regresa un `String` vacío, por lo que `println()` imprime una línea en blanco (inciso E). En la tercera impresión, `substring(7)` toma desde el índice 7 hasta el final, es decir, `"78\n"`, así que se imprime 78 seguido del salto de línea que agregó `indent()` (inciso D). Con todo esto presente, las respuestas correctas son los incisos A, D y E. Los distractores son claros: el inciso B (123) es para quien olvide que el índice final es exclusivo; el inciso C (7) es para quien piense que `substring(7, 7)` regresa el carácter del índice 7; y el inciso G es para quien piense que un inicio igual al final lanza una excepción. El inciso F es incorrecto, ya que todo compila. Los puntos clave son: `indent()` siempre agrega un salto de línea al final; `stripLeading()` solo quita el espacio inicial; y en `substring(inicio, fin)`, el fin es exclusivo, y si inicio y fin son iguales, el resultado es un `String` vacío, no una excepción.

Pregunta número 13

13. What is the result of the following code?

```
public class Lion {  
    public void roar(String roar1, StringBuilder roar2) {  
        roar1.concat("!!!");  
        roar2.append("!!!");  
    }  
    public static void main(String[] args) {  
        var roar1 = "roar";  
        var roar2 = new StringBuilder("roar");  
        new Lion().roar(roar1, roar2);  
        System.out.println(roar1 + " " + roar2);  
    }  
}
```

- A. roar roar
- B. roar roar!!!
- C. roar!!! roar
- D. roar!!! roar!!!
- E. An exception is thrown.
- F. The code does not compile.

Answer 13: En esta pregunta están evaluando la diferencia entre la inmutabilidad de String y la mutabilidad de StringBuilder, junto con el paso de parámetros a un método. Empezamos en el main: roar1 es el String "roar" y roar2 es un StringBuilder con "roar". Después llamamos a new Lion().roar(roar1, roar2); creamos una instancia y llamamos al método de instancia, todo correcto. En Java, los parámetros siempre se pasan por valor, pero cuando se trata de objetos, lo que se copia es la referencia; así que, dentro del método, roar1 y roar2 apuntan a los mismos objetos que en el main. Dentro del método, roar1.concat("!!!") crea un nuevo String "roar!!!", pero como String es inmutable y ese resultado nunca se asigna a ninguna variable, se pierde y el String original sigue siendo "roar". En cambio, roar2.append("!!!") modifica directamente el objeto StringBuilder, ya que es mutable, por lo que ahora contiene "roar!!!". De regreso en el main, se imprime roar1 + " " + roar2, que es "roar roar!!!". Por lo tanto, la respuesta correcta es el inciso B. El inciso D es la trampa para quien piense que concat() modifica el String original; el inciso C invierte el comportamiento de ambas clases y el inciso A es para quien piense que los cambios dentro del método nunca afectan al main. Los incisos E y F son incorrectos porque el código compila y corre sin problemas. Los puntos clave son: los métodos de String (concat, toUpperCase, replace, etc.) siempre regresan un nuevo String y nunca modifican el original; si no reasignamos el resultado, se pierde. En cambio, los métodos de StringBuilder modifican el mismo objeto, y ese cambio es visible desde cualquier variable que apunte a él.

Pregunta número 14

14. Given the following, which can correctly fill in the blank? (Choose all that apply.)

```
var date = LocalDate.now();  
var time = LocalTime.now();  
var dateTime = LocalDateTime.now();  
var zoneId = ZoneId.systemDefault();  
var zonedDateTime = ZonedDateTime.of(dateTime, zoneId);  
Instant instant = _____;
```

- A. `Instant.now()`
- B. `new Instant()`
- C. `date.toInstant()`
- D. `dateTime.toInstant()`
- E. `time.toInstant()`
- F. `zonedDateTime.toInstant()`

Answer 14: Aquí tenemos el tema de la clase `Instant` y cuáles clases de la API de fechas se pueden convertir a ella. Lo primero que debemos tener claro es que un `Instant` representa un punto exacto en la línea de tiempo universal (UTC). Para poder calcular ese punto, necesitamos tres datos: la fecha, la hora y la zona horaria. El primer inciso correcto es el A, ya que `Instant.now()` es el método estático que nos regresa el instante actual. El inciso B es incorrecto, porque las clases de `java.time` no tienen constructores públicos; siempre se crean con métodos estáticos como `now()` u `of()`, así que `new Instant()` no compila. El inciso C es incorrecto, ya que `LocalDate` solo tiene la fecha (no tiene hora ni zona), por lo que no tiene ningún método `toInstant()`. El inciso D es la trampa más fina: `LocalDateTime` tiene fecha y hora, pero no tiene zona horaria. Existe un método `toInstant(ZoneOffset)`, pero obligatoriamente necesita que le pasemos el offset; la versión sin argumentos no existe, así que este inciso no compila. El inciso E es incorrecto, porque `LocalTime` solo tiene la hora, sin fecha ni zona. El último inciso correcto es el F: `ZonedDateTime` tiene los tres datos necesarios (fecha, hora y zona), por lo que puede convertirse directamente con `toInstant()`. Con lo anterior dicho, las respuestas correctas son los incisos A y F. Los puntos clave son: para obtener un `Instant` necesitamos fecha, hora y zona; solo `ZonedDateTime` (u `OffsetDateTime`) puede usar `toInstant()` sin argumentos; `LocalDateTime` necesita que le pasemos el offset; y las clases de `java.time` nunca se crean con `new`.

Pregunta número 15

15. What is the output of the following? (Choose all that apply.)

```
var arr = new String[] { "PIG", "pig", "123"};  
Arrays.sort(arr);  
System.out.println(Arrays.toString(arr));  
System.out.println(Arrays.binarySearch(arr, "Pippa"));
```



Review Questions

- A. [pig, PIG, 123]
- B. [PIG, pig, 123]
- C. [123, PIG, pig]
- D. [123, pig, PIG]
- E. -3
- F. -2
- G. The results of `binarySearch()` are undefined in this example.

Answer 15: En este ejercicio estamos combinando el ordenamiento de arreglos con `Arrays.sort()` y la búsqueda binaria con `Arrays.binarySearch()`. Empezando con el ordenamiento, debemos recordar que los `String` se ordenan según su orden natural, que es lexicográfico basado en los valores Unicode de cada carácter. La regla que me gusta recordar es "números, mayúsculas, minúsculas": los dígitos (48 a 57) van antes que las letras mayúsculas (65 a 90), y estas van antes que las minúsculas (97 a 122). Por lo tanto, el arreglo ordenado queda como [123, PIG, pig], que es el inciso C. De ahí nos movemos a la búsqueda de "Pippa". Como el arreglo ya está ordenado, el resultado de `binarySearch()` está bien definido, así que el inciso G es incorrecto. "Pippa" no existe en el arreglo, por lo que tenemos que calcular dónde se insertaría. Si comparamos "Pippa" con "PIG", la primera letra 'P' es igual, pero en la segunda tenemos 'i' minúscula (105) contra 'I' mayúscula (73), así que "Pippa" va después de "PIG". Si comparamos "Pippa" con "pig", la 'P' mayúscula (80) es menor que la 'p' minúscula (112), por lo que "Pippa" va antes de "pig". Entonces, su punto de inserción es el índice 2. Cuando un elemento no se encuentra, `binarySearch()` regresa $-(\text{punto de inserción}) - 1$, es decir, $-(2) - 1 = -3$, que es el inciso E. Con todo,

esto, las respuestas correctas son los incisos C y E. El inciso F (-2) es la trampa para quien olvide restar ese 1 adicional, y los incisos A, B y D son ordenamientos incorrectos que ignoran la regla de Unicode. Los puntos clave son: los String se ordenan como números primero, luego mayúsculas y al final minúsculas; si `binarySearch()` no encuentra el elemento, regresa el negativo del punto de inserción menos 1; y el resultado solo es indefinido si el arreglo no está ordenado.

Pregunta número 16

16. What is included in the output of the following code? (Choose all that apply.)

```
var base = "ewe\nsheep\\t";
int length = base.length();
int indent = base.indent(2).length();
int translate = base.translateEscapes().length();

var formatted = "%s %s %s".formatted(length, indent, translate);
System.out.format(formatted);
```

- A. 10
- B. 11
- C. 12
- D. 13
- E. 14
- F. 15
- G. 16

Answer 16: Considero que esta pregunta pone a prueba nuestra atención al detalle para contar caracteres, sobre todo con las secuencias de escape. Empezamos con la variable `base = "ewe\nsheep\\t"`. Aquí tenemos que tener mucho cuidado: `\n` es un solo carácter (el salto de línea) y `\\` también es un solo carácter (una diagonal invertida), seguida de una letra `t` normal. Contando, tenemos `e`, `w`, `e`, salto de línea, `s`, `h`, `e`, `e`, `p`, diagonal y `t`, lo que nos da una longitud de 11 (inciso B). De ahí nos movemos a `base.indent(2)`. El método `indent()` separa el texto en líneas, agrega 2 espacios al inicio de cada una y además se asegura de que cada línea termine con un salto de línea, incluyendo la última. La primera línea queda como dos espacios + `"ewe"` + salto de línea, que son 6 caracteres. La segunda queda como dos espacios + `"sheep\t"` (7 caracteres, ya que la diagonal y la `t` son dos caracteres) + el salto de línea que se agrega al final, que son 10 caracteres. En total tenemos 16 (inciso G). Finalmente, ``translateEscapes()`` convierte las secuencias de escape escritas literalmente en su carácter real; la diagonal seguida de la `t` se convierte en un solo carácter de tabulación, por lo que la longitud baja de 11 a 10 (inciso A). La línea del `formatted()` simplemente sustituye los tres `%s`, y se imprime `"11 16 10"`. Con todo lo

anterior dicho, las respuestas correctas son los incisos A, B y G. Los distractores salen de errores de conteo: el 12 aparece si contamos \n como dos caracteres, el 15 si olvidamos el salto de línea que indent() agrega al final, y el 14 si olvidamos que se agregan espacios a ambas líneas. Los puntos clave son: cada secuencia de escape (\n, \t, \\) cuenta como un solo carácter; indent() agrega espacios a cada línea y siempre termina el texto con un salto de línea; y translateEscapes() convierte los escapes literales en su carácter real, reduciendo la longitud.

Pregunta número 17

17. Which of these statements are true? (Choose all that apply.)

```
var letters = new StringBuilder("abcdefg");
```

- A. `letters.substring(1, 2)` returns a single-character String.
- B. `letters.substring(2, 2)` returns a single-character String.
- C. `letters.substring(6, 5)` returns a single-character String.
- D. `letters.substring(6, 6)` returns a single-character String.
- E. `letters.substring(1, 2)` throws an exception.
- F. `letters.substring(2, 2)` throws an exception.
- G. `letters.substring(6, 5)` throws an exception.
- H. `letters.substring(6, 6)` throws an exception.

Answer 17: Los puntos clave son el método `substring()` de `StringBuilder` y sus reglas de índices. Lo primero que debemos recordar es que `substring()` en `StringBuilder` regresa un nuevo String y no modifica el contenido del `StringBuilder`. Las reglas son las mismas que en `String`: el índice de inicio debe ser menor o igual al índice final, el índice final debe ser menor o igual a la longitud, y el índice final es exclusivo. Nuestro `StringBuilder` contiene "abcdefg", que tiene una longitud de 7, con índices del 0 al 6. Empezando con `substring(1, 2)`, toma desde el índice 1 hasta el 2 sin incluirlo, es decir, solo la "b"; por lo tanto, regresa un String de un solo carácter y el inciso A es correcto, mientras que el inciso E es incorrecto porque no hay excepción. Con `substring(2, 2)`, el inicio es igual al final; esto es válido y regresa un String vacío, por lo que el inciso B es incorrecto (no es un solo carácter) y el inciso F también es incorrecto (no lanza excepción). Con `substring(6, 5)`, el índice de inicio es mayor que el final, lo cual rompe la regla y lanza una `StringIndexOutOfBoundsException`; por lo tanto, el inciso G es correcto y el inciso C es incorrecto. Finalmente, con `substring(6, 6)`, el inicio es igual al final, regresa un String vacío sin excepción, por lo que los incisos D y H son incorrectos. Con todo esto visto, las respuestas correctas son los incisos A y G. Los puntos clave son: si el inicio y el final son iguales, el resultado es un String vacío, nunca una excepción; si el inicio es mayor que el final, siempre hay excepción; y la cantidad de caracteres que regresa `substring(inicio, fin)` es exactamente `fin - inicio`.

Pregunta número 18

18. What is the result of the following code? (Choose all that apply.)

```
13: String s1 = ""  
14:     purr"";
```



216 Chapter 4 • Core APIs

```
15: String s2 = "";  
16:  
17: s1.toUpperCase();  
18: s1.trim();  
19: s1.substring(1, 3);  
20: s1 += "two";  
21:  
22: s2 += 2;  
23: s2 += 'c';  
24: s2 += false;  
25:  
26: if ( s2 == "2cfalse") System.out.println("==");  
27: if ( s2.equals("2cfalse")) System.out.println("equals");  
28: System.out.println(s1.length());
```

- A.** 2
- B.** 4
- C.** 7
- D.** 10
- E.** ==
- F.** equals
- G.** An exception is thrown.
- H.** The code does not compile.

Answer 18: En esta pregunta se combinan varios temas: los text blocks, la inmutabilidad de String, la concatenación con += y la diferencia entre == y equals(). Requerimos ir línea por línea. En las líneas 13 y 14 tenemos un text block; después de las comillas triples de apertura viene un salto de línea obligatorio, y en la

siguiente línea tenemos los espacios, la palabra purr y las comillas de cierre en la misma línea. Como el cierre está en la misma línea, no se agrega ningún salto de línea al final, y Java elimina la indentación incidental (los espacios del inicio), así que s1 vale simplemente "purr". En la línea 15, s2 es un String vacío. Las líneas 17, 18 y 19 son trampas: toUpperCase(), trim() y substring() regresan nuevos String, pero como nunca se reasignan, se pierden y s1 sigue valiendo "purr". En la línea 20, s1 += "two" sí reasigna la variable, así que ahora vale "purrtwo". De ahí pasamos a s2: en la línea 22, s2 += 2 nos da "2"; en la línea 23, s2 += 'c' nos da "2c"; y en la línea 24, s2 += false nos da "2cfalse". Recordemos que cuando uno de los operandos es String, cualquier tipo (int, char o boolean) se concatena sin problema, por lo que todo compila. En la línea 26, s2 == "2cfalse" es false, porque s2 se construyó en tiempo de ejecución y es un objeto distinto al literal del pool; no se imprime "==". En la línea 27, equals() compara el contenido, que es idéntico, así que se imprime "equals". Finalmente, en la línea 28 se imprime la longitud de "purrtwo", que es 7. Con todo esto presente, las respuestas correctas son los incisos C y F. Los incisos A, B y D salen de pensar que alguna de las líneas 17 a 19 modificó s1 o de contar mal el contenido del text block (por ejemplo, el inciso B es la longitud de "purr" si olvidamos la línea 20). El inciso E es la trampa del String pool, y los incisos G y H son incorrectos porque el código compila y corre sin errores. Los puntos clave son: los métodos de String que no se reasignan no cambian nada; en un text block se elimina la indentación incidental y no hay salto de línea final si el cierre está en la misma línea del contenido; y un String construido en tiempo de ejecución nunca es == a un literal, aunque tenga el mismo contenido.

Pregunta número 19

19. Which of the following fill in the blank to print a positive integer? (Choose all that apply.)

```
String[] s1 = { "Camel", "Peacock", "Llama"};
String[] s2 = { "Camel", "Llama", "Peacock"};
String[] s3 = { "Camel"};
String[] s4 = { "Camel", null};
System.out.println(Arrays._____);
```

- A. compare(s1, s2)
- B. mismatch(s1, s2)
- C. compare(s3, s4)
- D. mismatch (s3, s4)
- E. compare(s4, s4)
- F. mismatch (s4, s4)

Answer 19: En este problema tenemos los métodos Arrays.compare() y Arrays.mismatch(), así que primero tenemos que tener muy claras sus reglas. Arrays.compare() compara elemento por elemento y regresa un número negativo si

el primer arreglo es "menor", 0 si son iguales y un número positivo si el primero es "mayor"; si un arreglo es el prefijo del otro, el más corto es el menor, y un null se considera menor que cualquier valor. Por su parte, `Arrays.mismatch()` regresa el índice de la primera diferencia, o -1 si los arreglos son iguales; si uno es prefijo del otro, regresa la longitud del más corto. Empezando con el inciso A, `compare(s1, s2)`: en el índice 0 ambos tienen "Camel"; en el índice 1 comparamos "Peacock" contra "Llama", y como 'P' va después de 'L' en el alfabeto, el resultado es positivo; el inciso A es correcto. El inciso B, `mismatch(s1, s2)`, regresa 1, que es el índice de la primera diferencia y es un número positivo; también es correcto. El inciso C, `compare(s3, s4)`, es incorrecto: `s3` ("Camel") es prefijo de `s4` ("Camel", null), por lo que `s3` es el menor y el resultado es negativo. El inciso D, `mismatch(s3, s4)`, es correcto: la primera diferencia ocurre en el índice 1, porque `s3` ya no tiene más elementos, así que regresa 1. El inciso E, `compare(s4, s4)`, compara el arreglo consigo mismo y regresa 0, que no es positivo. Finalmente, el inciso F, `mismatch(s4, s4)`, regresa -1 porque no hay diferencias. Un detalle importante es que el null dentro de `s4` no provoca ningún `NullPointerException` en estos métodos. Con lo anterior dicho, las respuestas correctas son los incisos A, B y D. Los puntos clave son: `compare()` regresa negativo, 0 o positivo, y el arreglo más corto (prefijo) es el menor; `mismatch()` regresa el índice de la primera diferencia o -1 si son iguales; y debemos fijarnos que la pregunta pide un entero positivo, por lo que el 0 y el -1 quedan descartados.

Pregunta número 20

20. Note that March 13, 2022 is the weekend that clocks spring ahead for daylight saving time. What is the output of the following? (Choose all that apply.)

```
var date = LocalDate.of(2022, Month.MARCH, 13);
var time = LocalTime.of(1, 30);
var zone = ZoneId.of("US/Eastern");
var dateTime1 = ZonedDateTime.of(date, time, zone);
var dateTime2 = dateTime1.plus(1, ChronoUnit.HOURS);

long diff = ChronoUnit.HOURS.between(dateTime1, dateTime2);
int hour = dateTime2.getHour();
boolean offset = dateTime1.getOffset()
    == dateTime2.getOffset();
System.out.println("diff = " + diff);
System.out.println("hour = " + hour);
System.out.println("offset = " + offset);
```

- A. diff = 1
- B. diff = 2
- C. hour = 2
- D. hour = 3
- E. offset = true
- F. The code does not compile.
- G. A runtime exception is thrown.

Answer 20: Vuelve a aparecer el horario de verano, pero esta vez con operaciones sobre un `ZonedDateTime`. Vayamos línea por línea. Primero tenemos la fecha 13 de marzo de 2022, la hora 1:30 y la zona US/Eastern. A la 1:30 todavía no ha ocurrido el cambio de horario, así que `dateTime1` es una hora válida: 01:30 con offset -05:00 (horario estándar). De ahí pasamos a `dateTime2 = dateTime1.plus(1, ChronoUnit.HOURS)`; esto suma una hora real transcurrida. Normalmente llegaríamos a las 2:30, pero ese día los relojes saltan de las 2:00 a las 3:00, así que las 2:30 no existen; el resultado es 03:30 con offset -04:00 (horario de verano). Ahora nos dirigimos a las variables. `diff` usa `ChronoUnit.HOURS.between()`, que mide el tiempo real transcurrido entre los dos instantes, no la diferencia de las horas en el reloj; como solo sumamos una hora real, `diff` vale 1. `hour` obtiene la hora del reloj de `dateTime2`, que es 3. Finalmente, `offset` compara -05:00 contra -04:00; son diferentes, así que vale `false`. La salida es `diff = 1`, `hour = 3` y `offset = false`. Por lo tanto, las respuestas correctas son los incisos A y D. Los distractores están diseñados para caer en la trampa del reloj: el inciso B (`diff = 2`) sale de restar las horas del reloj (3 - 1), el inciso C (`hour = 2`) sale de ignorar el cambio de horario, y el inciso E sale de pensar que el offset nunca cambia. Los incisos F y G son incorrectos, ya que `plus(long, TemporalUnit)` es un método válido y el cambio de horario nunca lanza una excepción. Los puntos clave son: sumar horas a un `ZonedDateTime` suma tiempo real, por lo que, al cruzar el cambio de horario, la hora del reloj "salta"; `ChronoUnit.between()` siempre mide el tiempo real transcurrido; y en el cambio de primavera el offset cambia (en US/Eastern, de -05:00 a -04:00).

Pregunta número 21

21. Which of the following can fill in the blank to print `avaJ`? (Choose all that apply.)

```
3: var puzzle = new StringBuilder("Java");
```

```
4: puzzle._____;
```

```
5: System.out.println(puzzle);
```

- A. `reverse()`
- B. `append("vaJ$").substring(0, 4)`
- C. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length() - 1)`
- D. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length())`
- E. None of the above

Answer 21: En este ejercicio estamos evaluando los métodos de `StringBuilder` y, algo muy importante, el orden en que Java evalúa una cadena de métodos con sus argumentos. Partimos de que el `puzzle` contiene "Java" y que el objetivo es imprimir "avaJ". El primer inciso correcto es el A: `reverse()` invierte el contenido del mismo `StringBuilder`, dejándolo como "avaJ". El inciso B es una trampa de mutabilidad: `append("vaJ$")` modifica el objeto y lo deja como "Javava\$", pero después `substring(0, 4)` regresa un nuevo `String` ("Java") que nunca se asigna a nada; el `StringBuilder` sigue dejando "Javava\$", que es lo que se imprime. El siguiente inciso

correcto es el C. Hay que ir paso a paso: `append("vaJ$")` deja el contenido como "Javava\$" (8 caracteres); después, `delete(0, 3)` elimina los índices 0, 1 y 2 ("Jav"), dejando "avaJ\$" (5 caracteres). Aquí está la clave: Java evalúa primero el objeto sobre el que se llama el método (toda la cadena anterior) y después los argumentos, por lo que `puzzle.length()` se calcula cuando el contenido ya es "avaJ\$"; entonces, `puzzle.length() - 1` es 4, y `deleteCharAt(4)` elimina el '\$', dejando "avaJ". El inciso D es incorrecto: sigue los mismos pasos, pero `deleteCharAt(puzzle.length())` intenta borrar el índice 5, que no existe (el último índice válido es 4), y lanza una `StringIndexOutOfBoundsException`. Como sí hay respuestas correctas, el inciso E queda descartado. Con lo anterior dicho, las respuestas correctas son los incisos A y C. Los puntos clave son: en una cadena de métodos de `StringBuilder`, los argumentos se evalúan de izquierda a derecha y ven el estado actualizado del objeto; `substring()` regresa un `String` y no modifica el `StringBuilder`; y `deleteCharAt()` solo acepta índices de 0 a `length() - 1`.

Pregunta número 22

22. What is the output of the following code?

```
var date = LocalDate.of(2022, Month.APRIL, 30);
date.plusDays(2);
date.plusYears(3);
System.out.println(date.getYear() + " " + date.getMonth()
    + " " + date.getDayOfMonth());
```

218 Chapter 4 • Core APIs

- A. 2022 APRIL 30
- B. 2022 MAY 2
- C. 2025 APRIL 2
- D. 2025 APRIL 30
- E. 2025 MAY 2
- F. The code does not compile.
- G. A runtime exception is thrown.

Answer 22: Considero que esta pregunta es la hermana de la pregunta 11, ya que de nuevo la clave está en la inmutabilidad de las clases de la API de fechas. Tenemos que ir línea por línea. En la primera línea creamos un `LocalDate` con el 30

de abril de 2022, usando el enum `Month.APRIL`, lo cual es completamente válido. En la segunda línea, ``date.plusDays(2)`` calcula el 2 de mayo de 2022, pero regresa un nuevo objeto que nunca se asigna a ninguna variable, así que se pierde. En la tercera línea pasa exactamente lo mismo con `plusYears(3)`: calcula una nueva fecha que se descarta. La variable `date` sigue apuntando al objeto original. Finalmente, se imprime el año, el mes y el día de la fecha original: 2022 APRIL 30. Por lo tanto, la respuesta correcta es el inciso A. Los distractores están pensados para quien olvide la inmutabilidad: el inciso B (2022 MAY 2) sale de aplicar solo `plusDays(2)`; el inciso D (2025 APRIL 30) sale de aplicar solo `plusYears(3)`; el inciso E (2025 MAY 2) sale de aplicar ambas operaciones; y el inciso C es una combinación incorrecta. El inciso F es incorrecto porque, a diferencia de la pregunta 11, aquí todos los métodos existen en `LocalDate` (días y años sí son parte de una fecha), y el inciso G también es incorrecto porque el 30 de abril es una fecha válida. Los puntos clave son: todas las clases de `java.time` (`LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Instant`, `Period`, `Duration`) son inmutables, así que sus métodos `plus` y `minus` siempre regresan un nuevo objeto; si no lo reasignamos (`date = date.plusDays(2)`), la variable no cambia. Y siempre debemos revisar si cada método existe en la clase sobre la que se llama antes de calcular la salida.